

5 UVOD U C++

5.1 Strukturalno programiranje i C++

5.1.1 Odnos između Pascala i C-a

5.1.2 Odnos između C i C++

5 UVOD U C++

5.1 Strukturno programiranje i C++

U programiranju danas dominiraju dva osnovna koncepta : koncept strukturiranog programiranja i koncept objektno-orijentiranog programiranja. Pritom objektno orijentirano programiranje predstavlja nadgradnju koncepta strukturnog programiranja, obogaćujući ga novim mogućnostima i novim pristupom programiranju.

Najtipičniji primjer strukturno orijentiranog programskog jezika je Pascal. Pascal dosljednije od drugih programskih jezika primjenjuje tri osnovna koncepta strukturnog programiranja :

- stroga hijerarhijska struktura programa (što znači da je upotreba naredbe GO TO nepoželjna, ili barem svedena na minimum),
- odvajanje definicije podataka od njihove obrade (odnosno, postoje odjeljci u programu za definicije podataka i odjeljci za njihovu obradu) i
- potprogrami. Logičke cjeline unutar programa izdvajaju se u potprograme s točno određenom zadaćom, tako da se isti programski kod može pozivati iz različitih dijelova programa.

5.1.1 Odnos između Pascala i C-a

Osnovni koncepti Pascala prisutni su i u jeziku C, s tim da C dopušta veća odstupanja. C je jezik koji s jedne strane omogućuje programeru jednostavan pristup do elementarnih strojnih elemenata računala, s jedne strane, a s druge povećanu efikasnost programera s druge, zahvaljujući odmaku od krute strukture programa u Pascalu (osobito se to odnosi na strukture podataka).

Neke od osnovnih razlika između Pascala i C-a možemo vidjeti u slijedećoj tablici :

	Pascal	C
sekvenca	BEGIN-END	{ }
selekcija	IF, CASE	if, switch
iteracija	FOR, WHILE, REPEAT-UNTIL	for, while, do-while
potprogrami	procedure i funkcije	funkcije
slogovi	RECORD	struct
razlikovanje malih/velikih slova	NE	DA, ključne riječi obavezno malim slovima
aritmetički operatori	+ - * / div mod	+ - * / % unarni operatori
pokazivači	^	*
reference	funkcija ADDR	&
operator pridruživanja	:=	= operatori obnavljajućeg pridruživanja
relacijski operatori	= < > <> <= >=	== < > != <= >=
logički operatori	and or not	&& !
bitovni operatori	and or not shl shr	& ~ << >>
operator dodjele tipa	ne postoji (koriste se funkcije)	postoji
uvjetni operator	ne postoji	?
tip logičkih izraza	boolean (logički)	int (cjelobrojni)
komentar	{ i } ili (* i *)	/* i */

5.1.2 Odnos između C i C++

Kao što je Pascal najtipičniji strukturno-orijentirani programski jezik, tako je C++ (uz Javu, koja, međutim, nasljeđuje većinu koncepata i sintaksu od C++) najtipičniji objektno-orijentirani programski jezik. C++ predstavlja proširenje programskog jezika C, u kojeg uvodi koncept objektno-orijentiranog programiranja. Jezik C ostao je i dalje uključen u jezik C++, kao njegov podskup. Kao takav, C++ podržava oba programska koncepta : koncept strukturnog programiranja i koncept objektno-orijentiranog programiranja.

Usprkos tome što programi pisani u C-u rade i u C++, C++ u dosta slučajeva nudi drukčiji put rješavanja istih programskih zadataka.

5.1.2.1 Komentari

Osim oznaka `/* i */` iz C-a koje predstavljaju početak, odnosno kraj bloka koji predstavlja komentar, C++ uvodi i znak `//` koji označava red izvornog koda programa kao komentar. Na primjer :

```
{  
// Ovo je komentar!  
}
```

5.1.2.2 Dodjela tipa podataka

U C-u se dodjela tipa vrši tako da se tip navede u zagradi ispred naziva varijable, kao u slijedećem primjeru :

```
int i; float f;  
f = (float) i;
```

C++ dopušta i drugi način, po uzoru na poziv funkcije :

```
int i; float f;  
f = float (i);
```

5.1.2.3 Ulaz i izlaz

Ulaz i izlaz je u C++ riješen pomoću ulaznih i izlaznih tokova. Tokovi su klase definirane u standardnim bibliotekama, koje omogućuju komunikaciju programa s vanjskim jedinicama (npr. tipkovnicom i ekranom).

5.1.2.3.1 Terminalske ulaz/izlaz

Jedna od najočitijih razlika između C i C++ je zamjena standardne biblioteke *stdio* bibliotekom *iostream*. Biblioteka *iostream* zamjenjuje sve mogućnosti biblioteke *stdio*, odnosno, tri programska retka iz slijedećeg primjera daju identičan rezultat :

```
printf ("Danas je lijep dan.\n");           // C  
cout << "Danas je lijep dan.\n";           // kombinirano rješenje  
cout << "Danas je lijep dan." << endl;     // C++
```

Znakovi `<<` predstavljaju operator umetanja u tok.

Za formatiranje izlaza mogu se koristiti manipulatori, koji su definirani unutar biblioteke *iomanip*. Primjer :

```
float pi=3.14159;
cout << setprecision(4) << pi << endl; // ispisuje se 3.141
```

Ulazom se upravlja na sličan način, s tim da se koristi ulazni tok *cin* i operator izlučivanja iz toka (>>). Primjer :

```
scanf ("%f",&pi);      // C
cin >> pi;             // C++
```

5.1.2.3.2 Datotečni ulaz/izlaz

Umjesto biblioteke *stdio* koristi se biblioteka *fstream* (također se mogu koristiti *ifstream* i *ofstream*). Slijedeći primjer u C-u upisuje dva reda teksta u izlaznu tekstualnu datoteku :

```
FILE *dat;
dat = fopen ("tekst1.txt","wt");
fprintf (dat,"%s","Prvi red teksta!\n");
fprintf (dat,"%s","Drugi red teksta!\n");
fclose(dat);
```

Odgovarajući primjer u C++ :

```
fstream dat;
dat.open ("tekst.txt",ios::out);
dat << "Prvi red teksta!" << endl;
dat << "Drugi red teksta!" << endl;
dat.close();
```

U drugom primjeru definiran je datotečni objekt *dat*, iz klase *fstream*. Za otvaranje datoteke korišten je funkcijski član *open*. Datoteka je otvorena u modu *out* (izlazna datoteka). Tekst se u datoteku upisuje korištenjem operatora umetanja u tok (<<).

5.1.2.4 Deklaracije varijabli

Varijable se u C++ deklariraju na isti način kao i u C-u, ali mogu biti deklarirane u bilo kojem dijelu programskog koda. Primjer:

```
{
    int a;
    ... programski kod ...
    float b;
    ... programski kod ...
    char c;
    ... programski kod ...
}
```

5.1.2.5 Konstante

U C-u se za definiranje konstanti koristi pretprocesorska naredba *#define*. Na primjer:

```
#define UKUPNO 50
```

U tu svrhu u C++ se može koristiti i ključna riječ *const* :

```
const UKUPNO = 50;
```

Također, C++ dopušta da pojedini argumenti funkcija budu konstantni, time se sprečava promjena njihove vrijednosti unutar funkcije. Primjer :

```
void funkcija (const int a)
{
    a=10;          // Greška; ne može se promijeniti konstanta
}
```

5.1.2.6 Preopterećenje funkcije

C++ omogućuje da više funkcija ima isti naziv, pod uvjetom da su im liste parametara različite. Prema listi parametara kod poziva funkcije određuje se koja će funkcija biti pozvana. Primjer :

```
#include <iostream.h>
void funkcija(){
    cout << "Funkcija bez parametara!" << endl;
}
void funkcija(int a){
    cout << "Cjelobrojni parametar a = " << a << endl;
}
void funkcija(float b, float c){
    cout << "Realni parametar b = " << b << endl;
    cout << "Realni parametar c = " << c << endl;
}
void main(){
    funkcija();
    funkcija(10);
    funkcija(2.71,3.14);
}
```

Ispisuje se :

```
Funkcija bez parametara!
Cjelobrojni parametar a = 10
Realni parametar b = 2.71
Realni parametar c = 3.14
```

5.1.2.7 Podrazumijevani argumenti funkcija

Prilikom poziva funkcije potrebno je navesti listu argumenata koja odgovara listi argumenata u zaglavlju funkcije. C++ dopušta da se neki od argumenata u pozivu funkcije izostave, tako da se umjesto njih koriste podrazumijevane vrijednosti. Primjer:

```
#include <iostream.h>
void funkcija(int a, int b=5){
```

```

        cout << "a = " << a << endl;
        cout << "b = " << b << endl << endl;
    }
    void main(){
        funkcija (3,4);
        funkcija (10);
    }

```

Ispisuje se :

```

a = 3
b = 4
a = 10
b = 5

```

U drugom pozivu funkcije izostavljen je drugi argument, umjesto kojeg se koristi podrazumijevani (b=5).

5.1.2.8 Alokacija memorije

C++ zamjenjuje C-ovu funkciju za alokaciju memorije *malloc* i funkciju za dealokaciju memorije *free* s *new* i *delete*. *New* i *delete* omogućuju alokaciju korisnički definiranih tipova jednako kao i preddefiniranih. Primjer :

C:

```

int *pok;
pok = (int*)malloc(sizeof(int));
*pok = 10;
free (pok);

```

C++ :

```

int *pok;
pok = new int;
*pok = 10;
delete pok;

```

5.1.2.9 Deklaracije referenci

U C-u se često koriste pokazivači za prosljeđivanje argumenata funkcijama. U C++ može se koristiti referentni operator (&) u listi argumenata, što čini programski kod čišćim.

C:

```

void zamjena (int *a, int *b){
    int t = *a;
    *a = *b;
    *b = t;
}

```

```

void main(){
    int x = 3, y = 5;
    zamjena (&x, &y);
}

```

```

        printf ("%i %i\n",x,y);
    }
C++:
    void zamjena (int &a, int &b){
        int t = a;
        a = b;
        b = t;
    }
    void main(){
        int x = 3, y = 5;
        zamjena (x,y);
        cout << x << y << endl;;
    }

```

Također, moguće je referenci varijable pridružiti varijablu, kao u slijedećem primjeru :

```

int a = 0;
int &b = a;           // varijable a i b zauzimaju isti memorijski prostor
b = 5;
cout << a << endl;   // ispisuje se 5

```